

Tarea 1: Modelos de datos e información

Parte 1. Dato e información

Dato

Unidad mínima de información sin procesar. Es un hecho aislado (número, palabra, fecha) que por sí solo no tiene significado completo. Ejemplo: 25, "Juan", 15/08/2026.

Información

Resultado de procesar, organizar o relacionar datos, dándoles un significado útil para la toma de decisiones. Ejemplo: "Juan tiene 25 años y vive en la CDMX desde el 15/08/2026".

Características de la información

- **Relevancia:** es pertinente y útil para la decisión o propósito en cuestión.
- **Exactitud:** refleja la realidad de forma correcta, sin errores.
- **Oportunidad:** está disponible en el momento en que se necesita.
- **Compleitud:** abarca todos los aspectos necesarios, sin omisiones importantes.
- **Confiabilidad:** proviene de fuentes válidas y es consistente.
- **Claridad:** se presenta de forma comprensible para quien la recibe.
- **Verificabilidad:** puede comprobarse o confirmarse su validez.
- **Objetividad:** está libre de sesgos que distorsionen los hechos.
- **Accesibilidad:** puede obtenerse y consultarse cuando se requiera.
- **Valor agregado:** aporta algo que incrementa el conocimiento o reduce la incertidumbre.

Parte 2. Modelos de datos

1. Modelo orientado a objetos

Descripción

Modelo de base de datos que representa la información como objetos, tal como en la programación orientada a objetos. Cada objeto agrupa atributos (datos) y métodos (comportamiento), y puede relacionarse con otros objetos mediante referencias directas, en lugar de claves foráneas como en el modelo relacional.

Características

- Almacena la información como objetos, igual que en la programación orientada a objetos (atributos + métodos).
- Cada objeto tiene un identificador único (OID), independiente de sus valores.
- Soporta encapsulamiento, herencia y polimorfismo.
- Permite representar relaciones y estructuras complejas de forma directa, sin tablas intermedias.

Ventajas

- Elimina el "desajuste de impedancia" al mapear directamente con lenguajes de programación orientados a objetos.
- Maneja bien datos complejos, anidados y estructuras jerárquicas.
- Favorece la reutilización de código mediante herencia.
- Buen desempeño con datos multimedia y estructuras no tabulares.

Desventajas

- Menor estandarización que el modelo relacional (no hay un equivalente universal a SQL).
- Comunidad y soporte más reducidos.
- Curva de aprendizaje más pronunciada.
- Puede tener menor rendimiento en consultas complejas tipo reporte.

Casos de uso

- Sistemas CAD/CAM (diseño asistido por computadora).
- Aplicaciones multimedia y editores gráficos.
- Sistemas de información geográfica (GIS).
- Simulaciones científicas y telecomunicaciones.

Software que lo implementa

db4o, ObjectDB, Versant, InterSystems IRIS (antes Caché), Objectivity/DB, ZODB.

2. Modelo columnar (orientado a columnas)

Descripción

Modelo NoSQL que organiza y almacena los datos agrupándolos por columnas en lugar de por filas, agrupando columnas relacionadas en "familias de columnas". Está pensado para leer y agregar grandes volúmenes de datos de forma muy eficiente.

Características

- Almacena los datos agrupados por columnas en lugar de por filas.
- Organiza columnas relacionadas en "familias de columnas".
- Esquema flexible: no todas las filas necesitan tener las mismas columnas.
- Diseñado para escalar horizontalmente en clústeres distribuidos.

Ventajas

- Muy alto rendimiento en consultas analíticas y de agregación sobre grandes volúmenes.
- Mejor compresión de datos al almacenar valores similares juntos.
- Escalabilidad horizontal masiva.
- Lectura eficiente de subconjuntos de columnas sin cargar filas completas.

Desventajas

- No es óptimo para transacciones con muchas escrituras/actualizaciones puntuales (OLTP).
- Los joins complejos son costosos o poco soportados.
- Modelado de datos menos intuitivo, requiere pensar en patrones de consulta desde el diseño.

Casos de uso

- Big data y analítica a gran escala.
- Data warehousing.
- Series de tiempo e IoT.
- Procesamiento y análisis de logs.

Software que lo implementa

Apache Cassandra, Apache HBase, Google Bigtable, ClickHouse, Apache Druid, Amazon Redshift.

3. Modelo documental

Descripción

Modelo NoSQL que almacena la información en documentos semi-estructurados, típicamente en formato JSON o BSON, agrupados en colecciones. Cada documento puede tener una estructura distinta, lo que le da un esquema flexible y dinámico.

Características

- Almacena los datos en documentos semi-estructurados, normalmente en formato JSON o BSON.
- Cada documento puede tener una estructura distinta (esquema flexible/dinámico).
- Los documentos se agrupan en colecciones, análogas a las tablas relacionales.
- Permite estructuras anidadas (objetos dentro de objetos, arreglos, etc.).

Ventajas

- Gran flexibilidad de esquema, ideal para desarrollo ágil y datos cambiantes.
- Representación natural de datos jerárquicos sin necesidad de múltiples joins.
- Fácil de escalar horizontalmente mediante sharding.
- Buena integración con aplicaciones web y APIs (formato JSON nativo).

Desventajas

- Falta de estandarización entre distintos motores (cada uno con su propio lenguaje de consulta).
- Las transacciones que involucran varios documentos pueden ser más limitadas o costosas.
- Tiende a la redundancia de datos por la denormalización.

Casos de uso

- Sistemas de gestión de contenido (CMS).
- Catálogos de productos en e-commerce.
- Perfiles de usuario y personalización.
- Backends de aplicaciones móviles y APIs REST.

Software que lo implementa

MongoDB, CouchDB, Couchbase, Amazon DocumentDB, Firebase Firestore, RavenDB.

4. Modelo clave-valor

Descripción

Es el modelo NoSQL más simple: cada registro consiste en un par clave-valor, donde la clave funciona como identificador único y el valor puede ser prácticamente cualquier tipo de dato. El acceso se realiza siempre a través de la clave, como en una tabla hash.

Características

- Es el modelo NoSQL más simple: cada registro es un par clave-valor.
- El acceso a los datos se realiza únicamente a través de la clave (similar a una tabla hash).
- El valor puede ser de cualquier tipo: texto, número, JSON, binario, etc.
- No exige un esquema fijo para los valores almacenados.

Ventajas

- Rendimiento extremadamente alto en operaciones de lectura y escritura.
- Simplicidad de diseño e implementación.
- Escalabilidad horizontal muy sencilla (particionamiento por clave).
- Ideal como sistema de caché de alta velocidad.

Desventajas

- Consultas limitadas: no permite búsquedas complejas por el contenido del valor de forma nativa.
- No maneja bien relaciones entre distintos datos.
- Carece de capacidades analíticas avanzadas.

Casos de uso

- Sistemas de caché y gestión de sesiones web.
- Carritos de compra en e-commerce.
- Almacenamiento de configuraciones de aplicaciones.
- Colas de mensajes y datos temporales de alta velocidad.

Software que lo implementa

Redis, Amazon DynamoDB, Riak, Memcached, etcd, Berkeley DB.

5. Modelo orientado a grafos

Descripción

Modelo que representa la información como una red de nodos (entidades) conectados por aristas (relaciones), ambos con propiedades propias. Está optimizado para consultar y recorrer relaciones, ya que estas se almacenan físicamente como conexiones directas, sin necesidad de joins.

Características

- Representa los datos como nodos (entidades) y aristas (relaciones), ambos con propiedades.
- Está optimizado para recorrer y consultar relaciones entre entidades.
- No requiere joins costosos: las relaciones están almacenadas físicamente como conexiones directas.
- Permite modelar relaciones de cualquier grado de complejidad (redes, jerarquías, ciclos).

Ventajas

- Excelente desempeño en consultas de relaciones profundas y complejas.
- Más rápido que SQL con múltiples joins para este tipo de recorridos.
- Representación natural e intuitiva de redes y conexiones.
- Facilita el descubrimiento de patrones ocultos entre entidades.

Desventajas

- Curva de aprendizaje de lenguajes de consulta especializados (Cypher, Gremlin, SPARQL).
- No siempre es la mejor opción para datos tabulares simples.
- Escalar horizontalmente puede ser más complejo que en otros modelos NoSQL.

Casos de uso

- Redes sociales y análisis de conexiones entre usuarios.
- Sistemas de recomendación.
- Detección de fraude.
- Gestión de identidades y accesos, y análisis de rutas/logística.

Software que lo implementa

Neo4j, Amazon Neptune, ArangoDB, OrientDB, JanusGraph, TigerGraph.

Fuentes de información (formato APA, 7.^a edición)

Apache Software Foundation. (s. f.). *Overview*. Apache Cassandra Documentation. Recuperado el 18 de agosto de 2026, de <https://cassandra.apache.org/doc/latest/cassandra/architecture/overview.html>

IBM. (s. f.). *¿Qué son los datos?* IBM. Recuperado el 18 de agosto de 2026, de <https://www.ibm.com/mx-es/think/topics/data>

MongoDB. (s. f.-a). *Document database - NoSQL*. MongoDB. Recuperado el 18 de agosto de 2026, de <https://www.mongodb.com/resources/basics/databases/document-databases>

MongoDB. (s. f.-b). *What is an object-oriented database?* MongoDB. Recuperado el 18 de agosto de 2026, de <https://www.mongodb.com/resources/basics/databases/what-is-an-object-oriented-database>

Neo4j. (s. f.). *What is a graph database?* Neo4j Documentation. Recuperado el 18 de agosto de 2026, de <https://neo4j.com/docs/getting-started/graph-database/>

Redis. (s. f.). *Key-value databases*. Redis. Recuperado el 18 de agosto de 2026, de <https://redis.io/nosql/key-value-databases/>